Acknowledgment

We hereby take this opportunity to express our gratitude towards the people who helped us in completing our project **“Smart Test Cases Generator and SRS Validator”.**

We are especially thankful to our **Head Of Department Mr. Nishad Patel** and **Mentor Ms**. **Aishwarya Patil** for their valuable guidance, encouragement, and continuous support, which helped us successfully complete this project. Their suggestions and insights were extremely helpful at every stage of development.

We would also like to thank the faculty members of our department for providing us with the necessary knowledge and resources required for this work. Their guidance played an important role in shaping this project. We extend our gratitude to our team members for their cooperation and support during the project development.

Gunwant Mahendra Jadhav

Pranav Pankaj Jadhav

Hitika Avinash Mali

Manas Manoj Bhairao

Content

Chapter Number	Title	Page Number
	Abstract	i
	List of Figures	ii
	List of Tables	ii
1	Introduction	1
2	Literature Survey	3
2.1	Study of Existing System	3
2.2	Problem Identification	4
2.3	Problem Defination	4
3	Scope of the Project	6
4	Methodology	8
4.1	Requirements Gathering & Analysis.	8
5	Details of Designs, Working and Processes	10
5.1	System Diagram	12
6	Results and Applications	19
6.1	Home Page	19
6.2	Login Page	20
6.3	Register Page	22

6.4	Prompt Page	23
6.5	Invalid/Valid Module	26
6.6	Test Cases Page	29
6.7	Download Page	32
7	Testing and Evaluation	35
7.1	Introduction	35
7.2	Test Plan	41
7.3	Test Case For Login Functionality	43
8	Conclusion and Future Scope	48
9	Reference and Bibliography	49

Abstract

he Proposed system “Smart Test Cases Generator and SRS Validator” addresses the challenges faced in maintaining the quality of Software Requirement Specification (SRS) documents and the manual effort involved in generating test cases.

This system helps in validating the requirements effectively by detecting ambiguity, inconsistency, and incompleteness in the SRS document. We would like to express our sincere gratitude to our **Head of Department**,**Nishad Patel Sir**, and our **mentor, Aishwarya Patil**, for their valuable guidance and support. We are also thankful to our parents for their constant encouragement and motivation throughout this project.

The application is useful for developers, testers, and organizations facing issues related to unclear and error-prone requirements. It helps to analyze, validate, and structure requirements, and automatically generate test cases based on them, thereby improving software quality and reducing time and effort in the testing process.

The proposed system is developed using Frontend-React, Backend-Python, FastAPI and Mongodb.

List of figures

Fig No.	Title of figure	Page No.
5.1	System Diagram	12
5.2	Use Case Diagram	15
5.3	E-R Diagram	17
6.1	Home Page-Platform Introduction	19
6.2	Login Page— Secure Credentials based authentication	20
6.3	Register Page— User Creating new Account.	21
6.4	Prompt Page— Interaction with AI for generating output.	22
6.5	Valid/Invalid Page- system checks for issues like incomplete statements, missing keywords, and ambiguity.	23
6.6	Test Cases Page— Generates Valid Test	24
6.7	Downloads Page:- Stores History of Downloaded Content	25

List of tables

Table No.	Title of table	Page No.
1	Percentage of Defects	1
2	Manual Vs Automation testing	2
3	study of existing system	3
4	Collection list	17
5	UI Elements /Section -Login page	21
6	API’s Used -Login Page	21
7	UI Elements / Section -Registration Page	23

8	APIs Used -Registration Page	23
9	UI Elements / Section -Prompt Page	25
10	APIs Used -Prompt Page	25
11	UI Elements / Section -Validation Page	27
12	APIs Used -Validation Page	28
13	UI Elements / Section -Test Case Page	30
14	APIs Used -Test Case Page	31
15	UI Elements / Section -Download Page	33
16	APIs Used -Download Page	34
17	Test Cases On Login Functionality (Test Cases)	37
18	Test Cases On Login Functionality (Defect Report)	38
19	Test Case on Prompt Page	41
20	Defect Repot 1(Prompt Module)	42
21	Defect Repot 2(Prompt Module)	42
22	Test Cases For Login in WebDriver	43

Chapter 1

Introduction

In software development, the **Software Requirement Specification (SRS)** is one of the most important documents as it defines the functional and non-functional requirements of the system. However, in many cases, SRS documents contain ambiguous, incomplete, or inconsistent requirements. These issues can lead to misunderstandings, design errors, and increased development cost.

Traditionally, the validation of SRS documents and the creation of test cases are performed manually. This process is time-consuming, and requires significant effort from developers and testers. Moreover, if the requirements are not properly validated, the generated test cases may not accurately reflect the intended functionality of the system.

To overcome these challenges, this project proposes an **SRS Validator and Test Case Generator**. The system focuses on validating the SRS document to detect ambiguity and requirement issues, and then automatically generating test cases based on the validated requirements.

Overall, the proposed system aims to enhance requirement quality, support automated testing, and contribute to the development of reliable and efficient software systems.

Development Phase	**Percentage of Defects**
Requirement (SRS) Phase	50 – 56 %
Design Phase	27 – 30 %
Coding Phase	15 – 20 %

Table 1 Percentage of Defects

Studies in software engineering show that the majority of defects originate during the requirement phase. Errors in the SRS, such as ambiguity and incompleteness, contribute to more than half of the total defects. This shows the importance of validating the SRS document before coding and designing phase.

Aspect	Manual Process	Automated System
Requirement Validation	Time-consuming	Fast and efficient
Test Case Generation	Manual writing	Automatic generation
Accuracy	Depends on human effort	More consistent
Error Detection	Difficult	Easier with system
Time Required	High	Reduced

Table 2 Manual vs Automated Approach

In traditional software development, requirement validation and test case generation are performed manually, which is time-consuming and prone to human errors. The automated system improves this process by validating the SRS document and generating test cases efficiently. This reduces effort, improves accuracy, and enhances the overall software testing process.

Chapter 2

Literature Survey

2.1 Study of existing system

The existing system for software testing in academic and small-scale development environments is largely manual and fragmented. Test cases are typically written by testers based on Software Requirement Specification (SRS) documents using tools like Microsoft Word, Excel, or basic test management tools. This process is time-consuming, error-prone, and highly dependent on the tester's experience and understanding of the requirements.

In most cases, SRS validation is not automated. Testers manually interpret ambiguous or incomplete requirements, which leads to inconsistent test coverage. Additionally, there is no intelligent system that can analyze requirement quality (ambiguity, measurability, completeness) before generating test cases. This results in poor quality test cases that may miss critical scenarios.

Some tools exist for test management (like JIRA, TestRail), but they do not automatically generate test cases from raw SRS input. They require manual entry and structuring. Moreover, there is no integration between requirement validation and test case generation in these systems.

Sr no.	Existing System /Tool	Features	Limitations
1.	Manual Test Case writing.	Human written test cases based on SRS.	Time-consuming, error-prone, inconsistent.
2.	Excel-based Test Management	Stores Test Cases in structured form	No automation hard to maintain.
3.	Selenium Scripts (Manual Testing)	Automates Testing using scripts.	Requires coding knowledge
4.	JIRA/ Test Jail	Test case Management and tracking.	No automatic test case generation
5.	AI Tools(Basic GPT tools)	Generate test cases from text	Lack validation and structure
6.	Existing online Generators	SRS validation is not provided	Test cases are not proper due to invalid SRS.

Table 3 study of existing system

2.2 Problem Identification

Based on the study of the existing system, several key problems were identified:

- Manual Effort and Time Consumption
- Writing test cases manually from SRS documents requires significant time and effort.
- For large systems, this becomes inefficient and delays the testing phase.
- Ambiguous Requirements Handling
- Existing systems do not detect ambiguous terms like “fast”, “efficient”, or “user-friendly”, which leads to unclear and non-testable test cases.
- Lack of Requirement Validation
- There is no automated mechanism to validate whether a requirement is complete, measurable, and testable before generating test cases.
- Inconsistent Test Coverage
- Different testers may interpret the same requirement differently, leading to inconsistent and incomplete test coverage.
- No Intelligent Test Case Design
- Current approaches do not consider logic such as:
- Decision paths (if/else)
- Validation rules
- Calculation-based scenarios
- As a result, test cases are generic and not tailored to the requirement.
- Poor Integration Between SRS and Testing
- There is a disconnect between requirement documentation and test case generation. Testers must manually translate requirements into test cases.

2.3 Problem Definition

The main problem addressed in this project is the lack of an intelligent, automated system that can analyze Software Requirement Specifications (SRS) and generate high-quality, structured test cases. In the current software development lifecycle, the transition from requirements to testing is not automated, leading to inefficiencies, errors, and poor test coverage. There is a need for a system that can:

- Automatically extract valid requirements from raw SRS text
- Validate requirements based on quality attributes such as ambiguity, completeness, and measurability
- Identify logical patterns such as decision conditions, calculations, and validations
- Generate professional, structured, and comprehensive test cases
- Our project, TestGen AI, solves this problem by integrating:
- SRS Validation Engine.
- AI-based Requirement Analysis
- Smart Test Case Generation Engine
- This system reduces manual effort, improves accuracy, and ensures better test coverage by generating test cases directly from validated requirements.

Chapter 3

Scope of the Project

3.1 Scope of the project

The TestGen AI (Test Case Generator) application is designed to automate the process of generating software test cases from Software Requirement Specifications (SRS). This application helps developers, testers, and students to efficiently create structured, professional, and comprehensive test cases without manual effort.

The application analyzes SRS documents or user-input requirements and intelligently extracts functional and non-functional requirements. Based on this analysis, it generates multiple types of test cases such as positive, negative, boundary, and validation test cases.

Additionally, the application performs SRS validation by identifying ambiguity, completeness, and measurability issues in the requirements. It provides feedback and recommendations to improve the quality of the SRS document.

Following are the key features of this application:

1) SRS Input & Validation:

- Users can upload or paste SRS documents.
- The system validates requirements based on completeness, ambiguity, and measurability.
- Provides feedback and scoring for SRS quality.

2) Requirement Processing:

- Automatically extracts functional (FR) and non-functional (NFR) requirements.
- Splits complex requirements into atomic requirements.
- Identifies actors, actions, and constraints.

3) Automated Test Case Generation:

- Generates multiple types of test cases:
  - Positive test cases
  - Negative test cases
  - Boundary test cases
  - Validation test cases

- Creates structured test cases with:
    - Test Case ID
    - Test Steps
    - Test Data
    - Expected Result

4) Intelligent Test Design:

- Detects decision logic, calculations, and validations.
- Generates condition-based and branch test cases.
- Supports real-world scenario-based testing.

5) Reporting & Output:

- Displays generated test cases in professional tabular format.
- Allows export of test cases (PDF / Excel / Markdown).
- Provides validation checklist and improvement suggestions.

6) User Interface & Interaction:

- Simple and interactive UI for input and output.
- Back navigation support with preserved SRS and validation data.
- Real-time feedback and smooth workflow

Chapter 4

Methodology

4.1 Requirements Gathering and Analysis

For the development of the TestGen AI, we have adopted the Waterfall Model as the software development methodology.

The Waterfall Model is a linear and sequential approach where each phase must be completed before moving to the next phase. This model is suitable for our project as the requirements are clearly defined and stable.

SDLC Phases Applied :

Phase 1. Requirements Gathering and Analysis :

During this phase, we identified the requirements needed to develop an intelligent system that can validate SRS documents and generate professional test cases automatically.

4.1.1 User Requirements:

1. The system should allow users to input SRS (Software Requirement Specification) either by typing or uploading a file.
2. The system should provide a chat-based interface for interaction.
3. The system should display validation results for the given SRS.
4. The system should generate structured test cases from the validated SRS.
5. The system should allow users to view and navigate between prompt and test case pages.

4.1.2 SRS Validation Requirements:

The system should analyze and validate SRS requirements.

The system should detect:

- Ambiguous words
- Missing information
- Non-testable requirements

The system should provide:

- Issues list
- Verified Test Cases from SRS.

4.1.3 Test Case Generation Requirements:

The system should generate automatic test cases based on SRS.

The system should include:

- Test Case ID
- Objective
- Inputs / Steps
- Expected Result
- Actual Result
- Status

4.1.4 Navigation & UI Requirements:

The system should provide:

- Prompt page (input page)
- Test case result page

The system should allow:

- Back navigation from test case page to prompt page
- After Logout the Prompt Page should redirect to Home Page.
- From Download Module should redirect to the Prompt Page.

4.1.5 System Requirements:

- Operating System: Windows
- Frontend: React.js, HTML, CSS
- Backend: Python (Fast API)
- Database: SQLite / MySQL
- IDE: Visual Studio Code

4.1.6 Hardware Requirements:

- Processor: Intel i5
- RAM: 4 GB (minimum)
- Hard Disk: 500 GB (minimum)

Chapter 5

Details of designs, working and processes

Phase 2. System Design :

For TestGen AI – Automated Test Case Generator Application, the System Design phase involved creating a scalable architecture, efficient data handling mechanisms, and an interactive user interface tailored to validate SRS documents and generate professional test cases.

Here, we translated the gathered requirements into a structured system design that defines the application's architecture, modules, and workflow. Firstly, we defined the database schema to efficiently store SRS inputs, validation results, generated test cases, and user interaction history.

We identified the necessary data entities such as SRS requirements, validation metrics (ambiguity, completeness, measurability), generated test cases, and user session data. Based on the identified requirements, We defined the core functionalities of the application such as:

Based on the identified requirements, we designed the following key components:

- System Architecture:
  1. The system follows a full-stack architecture consisting of:
  2. Frontend (React.js) :- User interface for input, validation view, and test case display
  3. Backend (FastAPI- Python) :- Handles SRS processing, validation, and test case generation
  4. Processing Engine :- Custom AI-like logic for: Requirement extraction
  5. Test Case Generation.
- Database design:
  1. SRS input text
  2. Extracted requirements
  3. Validation results
  4. Generated test cases
  5. Chat/session data (for future enhancements)

- Core Functional Modules:

1. SRS Input Management
   - Allows users to upload or paste SRS documents for processing
2. SRS Validation Engine
   - Ambiguity detection
   - Completeness check
   - Measurability validation
3. Requirement Extraction Module
   - Splits and converts raw SRS into atomic requirements (FR/NFR)
4. Test Case Generation Engine:
   - Positive cases.
   - Negative cases.
   - Boundary cases.
   - Decision/logic-based cases.
5. Test Case Display Module:
   - Displays test cases in professional tabular format.
6. Navigation & State Management:
   - Maintains user state (SRS + validation) during navigation between pages.
7. History module:
   - Stores and display history of the generated test case.
8. Download Module:
   - Gives facility to download the test cases in excel/pdf format.

5.1 System Diagram

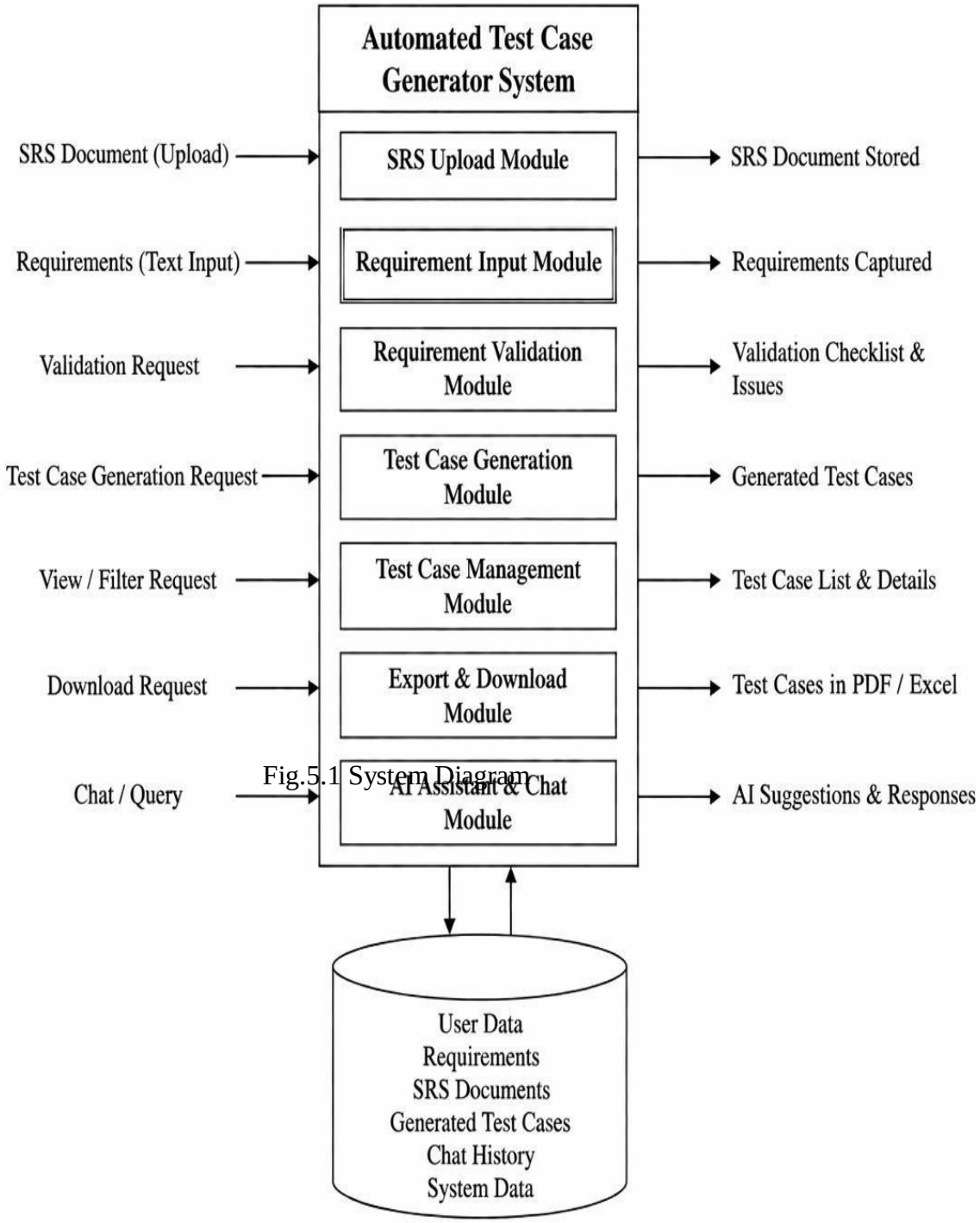


Fig.5.1 System Diagram

Fig 5.1 System Diagram

- **User Details:**

This module collects and manages information about the user of the system. It stores basic details such as username, email, and login credentials. It helps in authentication and maintaining user-specific data like chat history and generated test cases.

- **SRS Upload Module:**

The SRS Upload Module is responsible for handling Software Requirement Specification documents. It allows users to upload files in formats such as PDF, DOC, or TXT. The system processes the uploaded document and extracts relevant requirement data for further validation and test case generation.

- **Requirement Input Module:**

This module allows users to manually enter requirements in text form. It accepts structured or unstructured requirement descriptions provided by the user. These inputs are then stored and used for validation and test case generation.

- **Requirement Validation Module:**

This module analyzes the given requirements (from SRS or manual input). It checks for issues such as missing keywords, ambiguity, incomplete statements, and improper structure. It generates a validation checklist highlighting errors and improvements needed in the requirements.

- **Test Case Generation Module:**

The Test Case Generation Module automatically generates test cases based on validated requirements. It uses logic and AI techniques to create structured test cases including test steps, expected outputs, and conditions. This reduces manual effort and improves testing efficiency.

- **Test Case Management Module:**

This module allows users to view, manage, and organize generated test cases. Users can filter, edit, and review test cases before finalizing them. It helps in maintaining proper test documentation.

- **AI Suggestions Module:**

This module provides intelligent suggestions using AI. It helps users improve their requirements by suggesting better wording, missing scenarios, and additional edge cases. It enhances the quality of both requirements and test cases.

- **Download Module:**

The Download Module allows users to export generated test cases in formats like PDF or Excel. It helps in sharing and maintaining documentation for testing purposes.

- **Chat History Module:**

This module stores and manages the history of user interactions with the system. It keeps track of previous inputs, validations, and generated test cases, allowing users to revisit and reuse past data.

a. Use Case Diagram

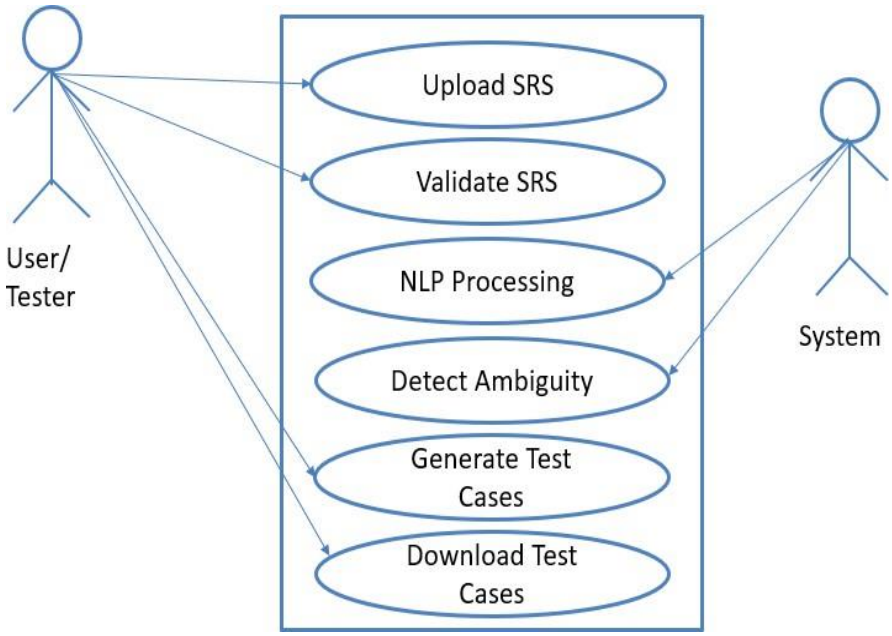


Fig. 5.2 Use Case Diagram

Actors:

• **User:** Represents the main user of the system (Tester / Developer / Student).

• **System:** Refers to the Automated Test Case Generator application.

• **Register:** The user can create a new account by providing basic details such as username, email, and password.

• **Login:** The user can log in to the system using valid credentials to access the application features.

• **Verification:** The system verifies the user's credentials and authenticates access before allowing further operations.

• **Handling Requirements:**

- **Upload SRS Document:** The user can upload an SRS document (PDF/DOC/TXT) into the system for processing.
- **Enter Requirements:** The user can manually input software requirements in text format.
- **View Requirements:** The user can view the entered or extracted requirements from the system.

• **Validate Requirements:** The system analyzes the requirements to detect errors, missing elements, or ambiguities.

- **View Validation Checklist:** The user can see a checklist showing validation results, including issues and suggestions.
- **Generate Test Cases:** The system automatically generates test cases based on validated requirements.
- **View Test Cases:** The user can view the generated test cases in a structured format.
- **Edit Test Cases:** The user can modify or refine generated test cases if needed.
- **AI Suggestions:** The system provides intelligent suggestions to improve requirement quality and coverage.
- **AI Test Cases:** The system generates advanced test cases using AI for better coverage and edge cases.
- **Download Test Cases:** The user can download test cases in formats like PDF or Excel.
- **View Chat History:** The user can view past interactions, including requirements and generated test cases.
- **Manage Chat History:** The user can manage or clear previous chats for better organization.

b. E-R Diagram

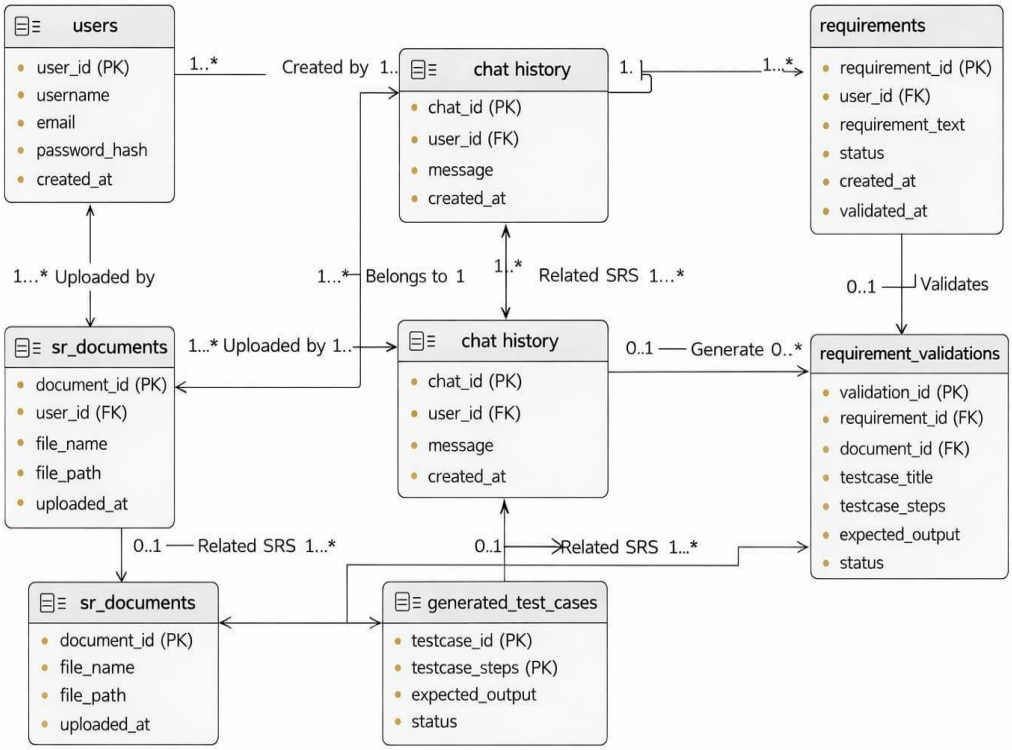


Fig. 5.3 ER Diagram

Sr no.	Collection Name	Description	fields
1.	users	Stores system users for authentication and access.	_id,username,email,password_hash, created_at
2.	sessions	Stores user session details for login management.	_id,user_id,file_name,file_path, uploaded_at
3.	srs_documents	Stores uploaded srs files and related metadata.	_id,user_id,session_token,created_at, expires_at
4.	requirements	Stores extracted or manually entered requirements.	id,user_id,issues_found, score,requirement_id, validated_at
5.	validation_results	Stores validation output for requirements	id,user_id,document_id,requirement_text, created_at
6.	testcases	Stores generated test cases from requirement.	id,user_id,testcase_title,status expected_output ,steps,
7.	ai_responses	Stores ai-generated outputs and suggestions	_id,user_id,prompt,response ,created_at
8.	chat_history	Stores user interactions and conversation history	_id,user_id,message,response,created_at

Table 4 Collection list

Chapter 6

Results and Applications

This chapter presents the results of the developed Automated Test Case Generator System through its major web pages and their functional descriptions. The screenshots demonstrate how users interact with the system to perform key operations such as uploading SRS documents, entering requirements, validating requirements, generating test cases, and downloading results.

Public Pages and User Access

The public-facing pages of the system are accessible before login. These pages introduce the platform and guide new users through the registration process while allowing existing users to log in securely. They serve as the entry point to the system and ensure proper authentication before accessing core functionalities.

6.1 Home Page:

Figure 6.1 shows the home page of the Rental Management System. It introduces the platform and provides navigation to key sections such as products, about, contact, login, and registration.

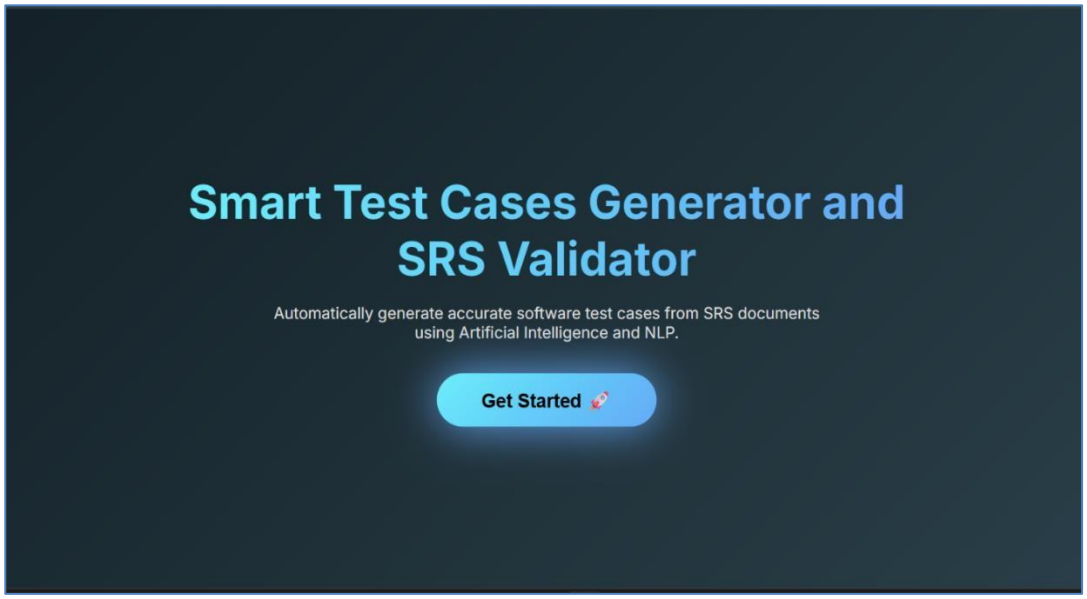


Figure 6.1: Home Page-Platform Introduction

- **Clear Project Purpose:** The heading “Smart Test Cases Generator and SRS Validator” along with the description clearly explains the functionality of the system using AI and NLP.
- **Strong Call-to-Action:** The “Get Started” button is prominently displayed at the center, encouraging users to begin using the application quickly.

6.2 Login Page

Figure 6.3 shows the registration page for new users. This page captures user details such as name, email, phone number, and password, then creates a new account in the system.

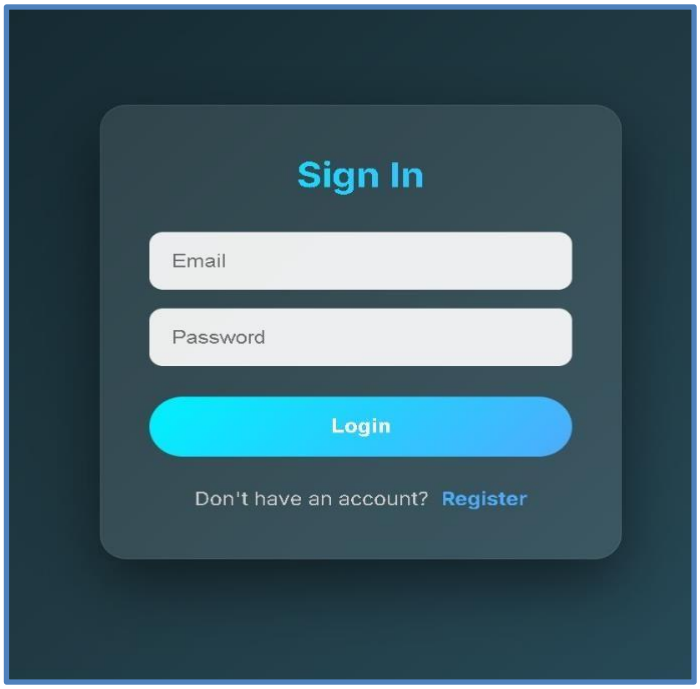


Figure 6.2: Login Page— Secure Credentials based authentication

The login page allows registered users to securely access the system using their email and password. After successful login, users are redirected to the main dashboard where they can upload SRS documents, validate requirements, and generate test cases.

The page includes input validation to prevent empty fields and shows error messages like “Invalid email or password” for incorrect details. It also ensures secure data handling and maintains user sessions during usage.

UI Element / Section	Explanation
Login Form Section	Displays the login interface in a clean, centered card layout where users enter their credentials to access the system. It includes a title, subtitle, and the form fields.
Input Fields	Includes two fields: • Email for the user's registered email address. • Password - for the user's account password.
Form Validation	Validates that the email is in a proper format and the password field is not empty. Shows inline error messages if validation fails before submitting the form.
Submit Button	A primary action button labeled "Login" that authenticates the user and grants access to the dashboard upon success.
Error Messages	Displays user-friendly error messages for invalid credentials, empty fields, or account-related issues (e.g., "Invalid email or password").
Navigation Option	A link below the form: "New user? Register here" that redirects new users to the registration page.

Table 5 UI Elements/Section -Login page

Method	Endpoint	Purpose
POST	/api/auth/login	Authenticates user credentials and returns a JWT token for secure access.
GET	/api/auth/me	Fetches details of the currently logged-in user using the JWT token.
POST	/api/auth/logout	Logs out the user by invalidating the current JWT token.

Table 6 API's Used -Login Page

6.3 Register Page:

Figure 6.3 shows the registration page for new users. This page captures user details such as name, email, roles, and password, then creates a new account in the system

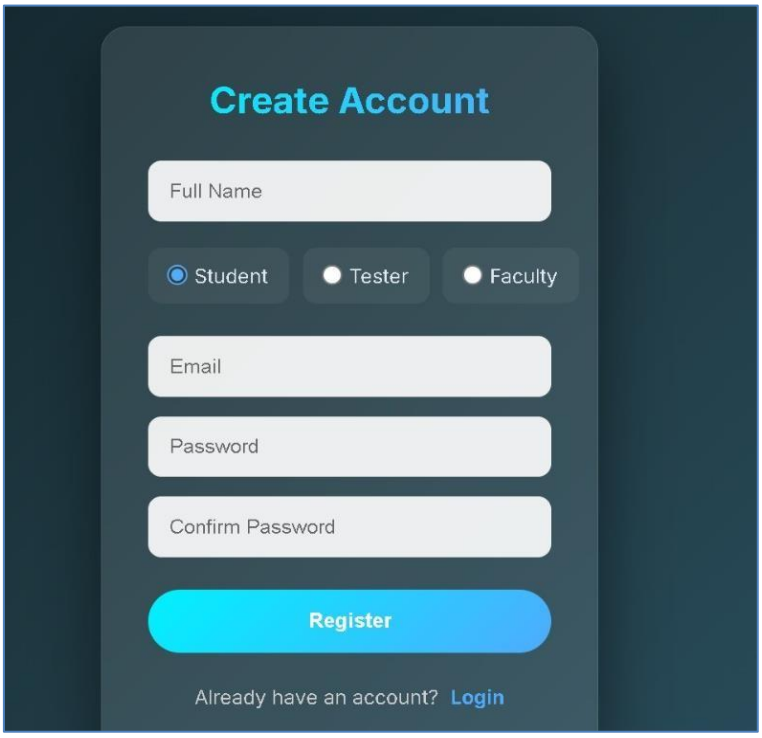


Figure 6.3: Register Page— User Creating new Account

UI Element / Section	Explanation
Registration Form Section	Thes centered, a dedicated form for new users to create an account. It placed at the center of the page inside a clean, responsive card layout that ensures better focus and ease of use. The form guides users
Page Title & Subtitle	A clear and welcoming title such as "Create Your Account" although a subtitle guided users through a simple.
Input Fields	Full Name, Email, Mobile Number, Password, Confirm Password, and Confirm Password,
	Checks for name format, valid

Form Validation	email, 10-digit mobile number, and a strong password, with user-friendly error messages for invalid inputs.
Terms & Conditions Checkbox	User must select a checkbox for users complete form and submit form to submit by login navigation link, below the form

Table 7 UI Section /Element -Registration page

Method	Endpoint	Purpose
POST	/api/auth/register	Registers a new user, creates an account, hashes JWT token for secure access.
POST	/api/auth/login	Authenticates a registered user for secure access.
POST	/api/auth/logout	Logs out user registered at login by invalidating.
GET	/api/auth/me	Fetches details of the currently logged-in user.

Table 8 APIs Used - Registration Page

The registration page includes proper input validation to ensure that all fields are filled correctly. It checks for valid email format, minimum password requirements, and prevents duplicate account creation using the same email address. If any invalid input is detected, appropriate error messages are displayed to guide the user.

After successful registration, the user can proceed to the login page and access the system features such as uploading SRS documents, validating requirements, and generating test cases.

6.4 Prompt Page:

Figure 6.4 shows the prompt page for the users. This page shows interface between the user and the system where the user uploads the requirements and the system generated the test cases by validating the SRS, In This Page provides options like:

1. Uploading docx/txt file or by give prompt.
2. New chat- for the chats that you want it to be special for each case.

3. Profile-(for downloads, logout, history)

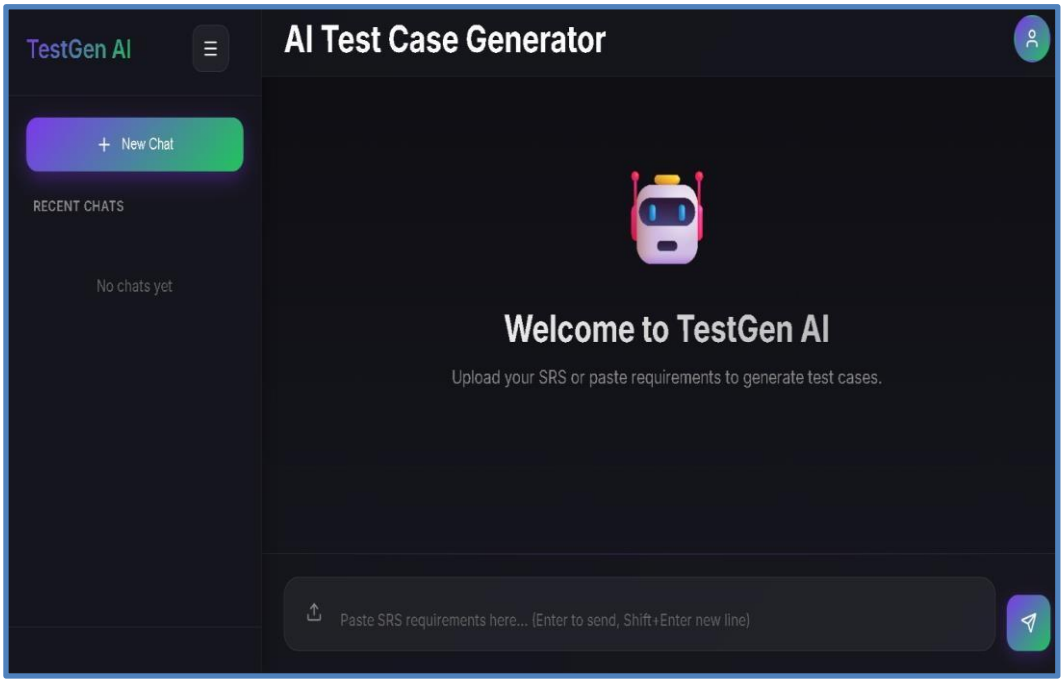


Figure 6.4: Prompt Page— Interaction with AI for generating output.

The Prompt Page is the main interface of the Automated Test Case Generator System, where users access core functionalities. It allows users to upload an SRS document or manually enter requirements in the input field.

After submission, the system performs requirement validation to identify issues such as missing keywords, incomplete statements, or ambiguity. The results are displayed as a checklist to help users improve their inputs.

Users can then generate test cases based on the validated requirements. These test cases can be viewed on a separate page and downloaded if needed. The page also provides AI suggestions and AI-generated test cases to improve quality and coverage.

Additional features include file attachment support, loading indicators, clear error messages, and system responses for a smooth user experience. It also maintains chat history, allowing users to review and reuse previous inputs and outputs.

UI Element / Section	Explanation
Prompt Input Area	A text area where users can enter and submit Software Requirements Specification (SRS) requirements as text prompts. It includes a title, subtitle, and the form fields. Users can input specific requirements for validation and test case generation.
File Upload Section	Allows users to upload SRS requirements documents including PDFs, DOCX files, and plain text files. A drag and-drop area and a browse button for file selection are available in this section, enabling users to upload their documents.
Submit Button	A prominent button labeled "Submit" is available that, when clicked, sends the user's entered prompt or uploaded file to the backend for analysis and processing. This triggers the validation of requirements and/or generation of test cases.
Chat Display Area	Shows an interactive chat-like interface that displays the user entered prompts and the system's responses. Allows users to view the validated results or generated test cases and interact with the Al to ask questions, get clarifications, and refine requirements in a conversational format.
Navigation Sidebar	Located on the left side of the page, displaying a clickable list of past chat sessions and offering a "New chat" button for initiating a new SRS prompt session. This sidebar provides

Table 9 UI Elements/Section -Prompt Page

Method	Endpoint	Purpose
POST	/api/upload_validate	Validates the uploaded Software Requirements Specification (SRS) document. Parses and analyzes the content to check for compliance, ambiguity, and compliance with SRS standards.
POST	/api/chats	Handles the storage and retrieval of user entered prompts and system generated responses. Maintains session state, stores chat history, and enables seamless continuous interaction for SRS validation and test case generation over single or multiple chat sessions.

Table 10 APIs Used -Prompt Page

6.5 Invalid/Valid Module:

Figure 6.5 shows the Validation Section based on the SRS. It appears after the user submits requirements or uploads a document, where the system checks for issues like incomplete statements, missing keywords, and ambiguity. The results are shown as a checklist with suggestions for improvement, helping users refine inputs. It also provides options to hide the checklist or proceed to test case generation, along with real-time feedback and smooth error handling

Figure 6.5: Valid/Invalid Page- system checks for issues like incomplete statements, missing keywords, and ambiguity.

The Validation Section appears on the prompt page after the user submits requirements or uploads an SRS document. It analyzes the quality of the input before test case generation.

The system checks for issues like incomplete statements, missing keywords, ambiguity, and improper structure, and displays the results as a checklist with improvement suggestions. This helps users refine requirements to make them clear and testable, ensuring accurate test cases.

It also provides options to hide the checklist or proceed with test case generation. Additionally, real-time feedback, along with proper error and loading handling, ensures a smooth user experience.

UI Element / Section	Explanation
Validation Results	A panel that centrally displays validation results of the submitted SRS requirements. It provides feedback on the quality and compliance of the requirements. It offers visual feedback, categorizing the requirements as valid, ambiguous or problematic using color-coded indicators.
	A dynamic and detailed checklist

Checklist Display	that shows the content of validation in bullet points, grouped into sections. • Missing Keywords - Keywords expected in quality SRS requirements but not present. • Ambiguities - Requirements that may be unclear or open to multiple interpretations.
Submit Button	A prominent button labeled "Submit" is available that, when clicked, sends the user's entered prompt or uploaded file to the backend for analysis and processing. This triggers the validation of requirements and/or generation of test cases.
Highlight Section	A text highlight feature that visually marks problematic phrases, ambiguities, and issues within the submitted requirements. It uses color-coded highlights to make these problem areas, easily distinguishable within the text, enabling immediate re-view and correction.
Action Buttons	A set of interactive buttons located below the validation panel, providing options to take action on the results: Show Testcases Jumps to test case generation screen based on validated requirements. Allows users to download a report of the Download Report validation findings (PDF). Al Refinement Provides Al-driven options to refine and revise requirements process.

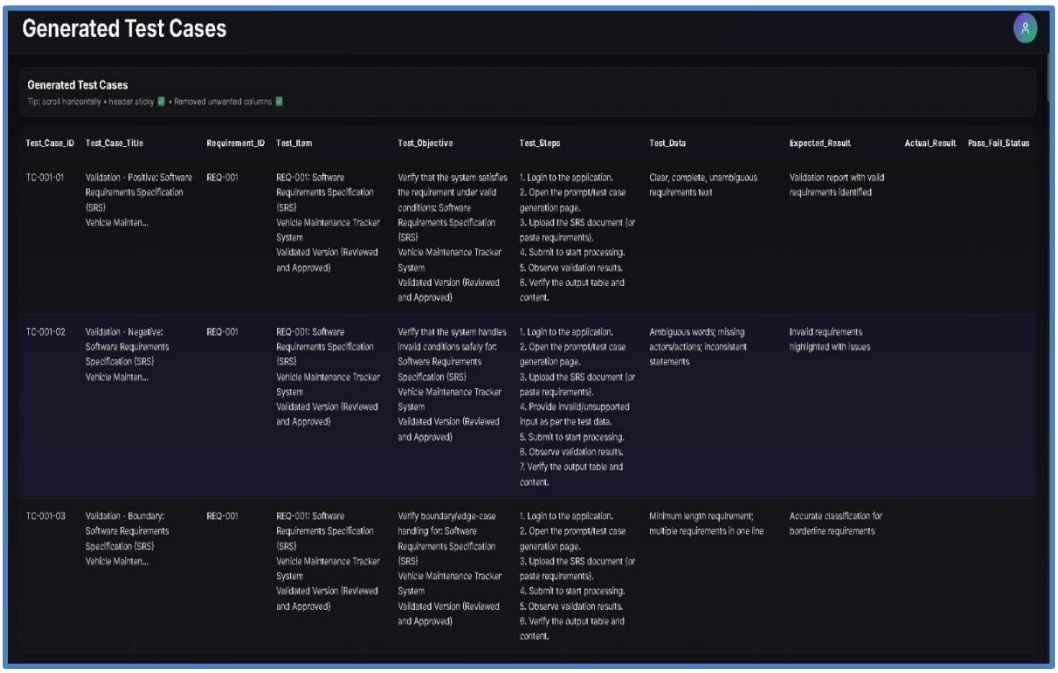
Table 11 UI Elements/Section -Validation Page

Method	Endpoint	Purpose
POST	/api/upload_validate	Validates and analyzes Software Requirements Specification (SRS) input submitted by the user. Parses documents including PDF, DOCX, and plain text files. Performs detailed analysis to identify missing keywords, ambiguities, problematic phrases, and structural issues. Generates a structured report with validation results.

Table 12 APIs Used -Validation Page

6.6 Test Cases Page:

Figure 6.6 shows the Test Cases Page of the Automated Test Case Generator System. This page displays the test cases generated from the validated requirements in a structured format. It acts as an interface where the system presents details such as test case ID, objective, steps, and expected output. The page allows users to review all scenarios and provides options to download the test cases in formats like PDF or Excel for easy storage and sharing.



Generated Test Cases

Generated Test Cases

Tip: scroll horizontally • header sticky ✅ • Removed unwanted columns ✅

Test_Case_ID	Test_Case_Title	Requirement_ID	Test_Item	Test_Objective	Test_Steps	Test_Data	Expected_Result	Actual_Result	Pass_Fail_Status
TC-001-01	Validation - Positive: Software Requirements Specification (SRS) Vehicle Mainten...	REQ-001	REQ-001: Software Requirements Specification (SRS) Vehicle Maintenance Tracker System Validated Version (Reviewed and Approved)	Verify that the system satisfies the requirement under valid conditions: Software Requirements Specification (SRS) Vehicle Maintenance Tracker System Validated Version (Reviewed and Approved)	1. Login to the application. 2. Open the prompt/test case generation page. 3. Upload the SRS document (or paste requirements). 4. Submit to start processing. 5. Observe validation results. 6. Verify the output table and content.	Clear, complete, unambiguous requirements text	Validation report with valid requirements identified		
TC-001-02	Validation - Negative: Software Requirements Specification (SRS) Vehicle Mainten...	REQ-001	REQ-001: Software Requirements Specification (SRS) Vehicle Maintenance Tracker System Validated Version (Reviewed and Approved)	Verify that the system handles invalid conditions safely for: Software Requirements Specification (SRS) Vehicle Maintenance Tracker System Validated Version (Reviewed and Approved)	1. Login to the application. 2. Open the prompt/test case generation page. 3. Upload the SRS document (or paste requirements). 4. Provide invalid/unsupported input as per the test data. 5. Submit to start processing. 6. Observe validation results. 7. Verify the output table and content.	Ambiguous words; missing actors/actions; inconsistent statements	Invalid requirements highlighted with issues		
TC-001-03	Validation - Boundary: Software Requirements Specification (SRS) Vehicle Mainten...	REQ-001	REQ-001: Software Requirements Specification (SRS) Vehicle Maintenance Tracker System Validated Version (Reviewed and Approved)	Verify boundary/edge-case handling for: Software Requirements Specification (SRS) Vehicle Maintenance Tracker System Validated Version (Reviewed and Approved)	1. Login to the application. 2. Open the prompt/test case generation page. 3. Upload the SRS document (or paste requirements). 4. Submit to start processing. 5. Observe validation results. 6. Verify the output table and content.	Minimum length requirement; multiple requirements in one line	Accurate classification for borderline requirements		

Figure 6.6: Test Cases Page— Generates Valid Test

The Test Cases Page displays test cases generated from validated requirements in a structured and organized format. After validation, the system automatically generates test cases and redirects the user to this page. Each test case includes details like test case ID, objective, steps, and expected output, making them clear and easy to use.

The page allows users to review all test cases, including edge cases, to ensure complete coverage and accuracy. It helps in verifying the correctness and reliability of the testing process. Additionally, the page provides options to download test cases in formats like PDF or Excel for easy storage and sharing, along with smooth navigation and a user-friendly interface.

UI Element / Section	Explanation
Testcases Display Table	A structured table that displays the generated test cases derived from the validated SRS requirements. The table is formatted with rows representing test cases, each including columns for Testcase ID, Steps, Expected Output, Status, and optionally Priority and Notes.
Columns Section	A dynamic and detailed checklist that shows the content of validation in bullet points: • Testcase ID A unique identifier for each test case. • Steps The specific steps required to execute the test case. • These are detailed in a step-by-step format. • Expected Output The anticipated result after executing the steps of the test case. • Status Indicates the state of the test case (e.g. Pending, Pass/Fail). • Optional: Priority Shows the importance level of the test case (High, Medium, Low). • Optional: Notes Additional comments or notes about the test case for context.
Navigation Controls	Buttons or links at the top of the testcases table that let users navigate:- • Back to Prompt - Returns to the prompt input section and previous chats. • Show SRS Requirements- Displays the original SRS requirements alongside the test cases. • Navigation Arrows -

	Pagination links to navigate through multiple pages of test cases, enabling users to browse through the generated test case list if it spans multiple pages.
Download Option	A prominent button, typically labeled "Download," located below or above the testcases table, that provides users with the ability to download the test cases in a structured document format such as XLSX, CSV, or DOCX. This allows for easy sharing, documentation, and integration into testing plans.

Table 13 UI Elements/Section – Test Cases Page

Method	Endpoint	Purpose
GET	/api/testcases (id)	Fetches the generated test cases based on the relevant requirements provided. Returns a list of structured test cases that are organized based on the provided SRS input.
POST	/api/ai/testcases	Generates and provides a list of test cases dynamically using Al based on the provided SRS requirements as input. The Al analyzes the SRS and automatically crafts appropriate and comprehensive test cases based on the SRS requirements.

Table 14 APIs Used -Test Cases Page

6.7 Download Page:

Figure 6.7 shows the Download Page of the Automated Test Case Generator System. This page provides an interface where users can view and download generated test cases. It displays a list of files with details such as file name and generation time, and offers options to download them in formats like PDF or Excel for easy storage and sharing.

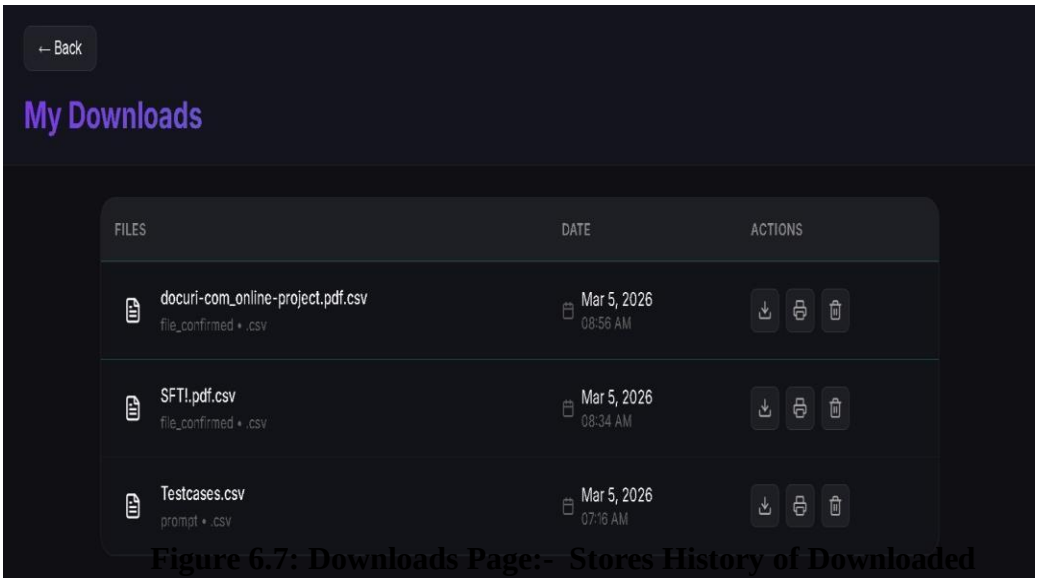


Figure 6.7: Downloads Page:- Stores History of Downloaded Content

The Download Page allows users to access and download generated test cases in a convenient format. It displays a list of files created from validated requirements and test case generation.

Users can view and download files in formats like PDF or Excel, making them suitable for documentation, sharing, and analysis. Each file is clearly labeled with details such as file name and generation time for easy identification.

The page provides quick access to downloads without needing to regenerate test cases. It helps users store, share, and reuse test cases efficiently, ensuring a smooth and practical user experience.

UI Element / Section	Explanation
Page Heading	A clear heading at the top, e.g., "Download Test Cases," with a short subtitle explaining that users can export the generated test cases in multiple formats (XLSX, CSV, DOCX).
Format Selector	dropdown or radio button group to choose the file format: • XLSX Microsoft Excel format. • CSV Comma-Separated Values • DOCX - Microsoft Word document
Filter & Customization (Optional)	Optional filters to customize the download: • Testcase ID Range - Enter start & end ID to download specific range (e.g., TC-001 to TC-050). • Status Filter - Multi-select (All, Pending, Pass, Fail). • Priority Filter - Multi-select (High, Medium, Low). • Module Filter Select specific module (Prompt, Validation, etc.). • Include Headers - Toggle to include column headers in CSV/XLSX.
Download Button	A primary button labeled "Download" that triggers the export of test cases based on the selected format and filters. The button displays a loading state while the file is being prepared.
Progress & Success Message	A small inline progress indicator (spinner or progress bar) and a success toast message after download starts, e.g., "Download started. Your file will be saved

	shortly."
Recent Downloads (Optional)	A collapsible section showing the last 5 downloaded files with details: File Name, Format, Date & Time, Record Count, and a quick "Download Again" action.
Navigation Controls	Back to Testcases Button to return to the Testcases page. Home / Dashboard - Quick link to go back to the main dashboard.

Table 15 UI Elements/Section-Download Page

Method	Endpoint	Purpose
GET	/api/testcases/export/config	Fetches available export formats and user permissions for download.
POST	/api/testcases/export	Generates and initiates the download of test cases based on selected format and applied filters. Returns a secure download URL or file stream.
GET	/api/testcases/export/{jobld}/status	(Optional) Checks the status of an ongoing export job using the job'ld. Returns progress and completion status.
GET	/api/testcases/export/history	Fetches the list of recently downloaded files with metadata (name, format, date, record count).
DELETE	/api/testcases/export/history/{id}	(Optional) Deletes a specific export history record by ID.

Table 16 APIs Used -Download Page

Chapter 7

Testing and Evaluation

7.1 Introduction

This chapter describes the testing activities performed for the Automated Test Case Generator System. Testing plays a crucial role in ensuring that the system functions accurately, efficiently, and securely while handling various user inputs and AI-generated outputs.

The system is designed to allow users to input Software Requirement Specifications (SRS), validate them using Artificial Intelligence, generate structured test cases, and download the results. Since the system integrates AI-based processing with user interaction, proper testing is required to ensure reliability, correctness, and performance.

The objective of this testing phase is to verify that all modules of the system operate correctly under different scenarios. The system should allow valid users to log in successfully, accept and process SRS inputs, generate validation results and test cases, and allow users to download outputs. It must also handle invalid inputs, prevent unauthorized access, and display appropriate error messages.

Testing was performed using both positive scenarios (valid login, valid prompt input, successful test case generation) and negative scenarios (invalid inputs, empty fields, API errors, unauthorized access).

Testing Tools Used

- **Selenium IDE** – Used for UI-based testing and quick automation.
- **Selenium WebDrive**r – Used for advanced automation and validation.
- **Postman** – Used for API testing.
- **Browser** Developer Tools – Used for debugging frontend issues.

Out of scope for this chapter:

Login Module, Defect Report,Test Plans.

7.2 Test Plan

The test plan defines the objectives, scope, testing environment, tools, entry and exit criteria, and risk factors associated with the login module testing cycle of the TestGen AI-Smart TestCases Generator and SRS Validator This test plan has been prepared following the IEEE standard format to ensure a well-structured and standardized documentation approach

- **TEST PLAN (LOGIN MODULE):**

Test Plan: TestGen AI-Smart Test Cases Generator and SRS Validator.

Version: 1.0

Test Plan ID: TP_LOGIN_01

1. Introduction

This test plan is designed to verify the functionality of the Login Module. It ensures that registered users can log in using valid credentials and that invalid login attempts are properly handled. It also ensures secure authentication and proper navigation after login.

2. Test Items

- Login form UI (email, password fields)
- Login button functionality
- Error message handling
- Authentication logic
- Session management

3. Features to be Tested

- Login with valid credentials
- Login with invalid credentials
- Empty field validation
- Invalid email format validation
- Password validation
- Navigation after login

4. Features Not to be Tested

- Prompt processing module
- Validation module
- Test case generation module
- Download module

5. Approach

- Functional testing will be performed
- Both positive and negative test cases will be executed
- Manual testing and Selenium automation will be used
- API testing using Postman

6. Pass / Fail Criteria

- All critical test cases must pass
- No high severity defects should remain
- Expected and actual results must match

7. Suspension Criteria

- Testing will be stopped if critical issues occur and resumed after fixing

8. Test Deliverables

- Test plan document
- Test case document
- Defect report
- Test summary report

9. Test Environment

- OS: Windows 11
- Browser: Chrome, Edge
- Frontend: React
- Backend: FastAPI
- Database: MongoDB

10. Risks

- Authentication failure
- Session handling issues
- Security vulnerabilities.

Test Cases On Login functionality:-

Tc_Id	Objective	Inputs/ Steps	Expected Output	Actual Output	Status
TC_LGN_001	Login with valid data	Valid email & password	Valid email & password	Successful	Pass
TC_LGN_002	Login with invalid password	Wrong password	Show error message	Error shown	Pass
TC_LGN_003	Login with invalid email	Wrong email	Show user not found	Error shown	Pass
TC_LGN_004	Empty Fields	Blank Inputs	Show Validation messages	Message shown	Pass

TC_LGN_005	Invalid email format	abc123	Show invalid email error	Error Shown	Pass
TC_LGN_006	Empty password	Only email entered	show password required	Message shown	Pass
TC_LGN_007	Multiple attempts	Wrong password repeatedly	Restrict login	Not restricted	Fail

Table 17 Test Cases On Login functionality(TestCases)

- **Defect Report:-**

Field	**Details**
Defect ID	DR_LGN_001
Project Name	Smart Test Cases Generator and SRS Validator.
Project Name	Web Application
Release Version	v1.0
Module	Login Module
Detected Build Version	v1.1
Summary	Unlimited login attempts allowed
Description	During testing, it was observed that the system does not restrict multiple failed login attempts. Users can continuously attempt login without any limitation, which may lead to security risks such as brute-force attacks.
Actual Result	Unlimited login attempts allowed
Expected Result	System should restrict attempts or show warning
Severity	High
Priority	High
Status	Open

Table 18 Test Cases On Login functionality(Defect Report)

- **TEST PLAN (PROMPT MODULE):**

Test Plan: Automated Test Case Generator System

Version: 1.0

Test Plan ID: TP_PROMPT_01

1. Introduction

This test plan is designed to verify the functionality of the Prompt Page Module of the Automated Test Case Generator System. The prompt page acts as the core interface where users input Software Requirement Specifications (SRS) to generate validation results and test cases using Artificial Intelligence. The purpose of this testing is to ensure that the system correctly accepts user input, validates it, processes it through the backend AI services, and provides accurate outputs such as validation feedback and generated test cases.

This module is critical because it directly interacts with AI processing, and any failure may impact the entire workflow of the system.

2. Test Items

- Prompt input field (textarea)
- Submit / Generate buttons
- AI Suggestion button
- AI Testcase button
- Validation response display
- Error messages for invalid inputs
- Loading indicator (AI processing)
- Session storage handling

3. Features to be Tested

- Submission of valid SRS input
- Handling of empty prompt input
- Handling of large and complex SRS inputs
- Special characters and formatting support
- AI integration (API call handling)
- Display of validation results
- Navigation to test case page

4. Features Not to be Tested

- Login and authentication module
- Database internal structure
- External API internal working
- Download functionality

5. Approach

- Functional testing will be performed
- Positive and negative test cases will be executed
- Manual testing for UI validation
- Selenium automation for repeated scenarios
- API testing using Postman
- Edge case testing for large input

6. Pass / Fail Criteria

- All major functionalities must work correctly
- System should not crash on invalid inputs
- Expected results must match actual outputs
- No high severity defects should remain

7. Suspension Criteria and Resumption

- Testing will be suspended if API fails or system crashes
- Testing will resume after fixing issues

8. Test Deliverables

- Test plan document
- Test case document
- Defect report
- Test summary report

9. Test Tasks

- Prepare test cases
- Execute test cases
- Identify defects

- Retest fixed defects
- Document results

10. Test Environment

- OS: Windows 11
- Browser: Google Chrome
- Frontend: React
- Backend: FastAPI
- Database: MongoDB
- Tools: Selenium, Postman

11. Responsibilities

- QA Engineer: Execute test cases
- Developer: Fix defects
- Project Guide: Review

12. Scheduling

- Testing will be completed before final submission deadline

13. Risks

- API failure (Gemini AI)
- Slow response for large input
- Incorrect validation results
- UI/UX issues

14. Approvals

- Project Guide and Team Members

Tc_Id	Objective	Inputs/ Steps	Expected Output	Actual Output	Status
TC_PRM_001	Upload Valid Input	Proper SRS= Upload	Process successfully Executed	Success	Pass
TC_LGN_002	Upload Empty input	Blank	Show error message	Error shown	Pass
TC_LGN_003	Upload Large input	Long SRS=versi-on3.txt	Handle properly	Success	Pass

TC_LGN_004	API call For Generating Outputs in table	Valid Input	Al response generated	Success	Pass
TC_LGN_005	Invalid Special chars format Input	Upload Symbols	Show invalid email error	Error Shown	Pass

Table 19 Test Cases On Prompt Page

- **Defect Report 1:-**

Field	**Details**
Defect ID	DR_PRM_001
Project Name	Smart Test Cases Generator and SRS Validator.
Project Name	Web Application
Release Version	v1.0
Module	Prompt Module
Detected Build Version	v1.1
Summary	System allows submission of empty prompt without validation
Description	During testing, it was observed that the system allows users to submit an empty prompt without performing validation. When the user clicks the submit button without entering any SRS content, the system still processes the request and sends it to the backend. This results in unnecessary API calls and may generate incorrect or empty responses. It also negatively impacts user experience as users are not guided properly.
Actual Result	System processes empty input
Expected Result	System should block submission and show error message
Severity	Medium
Priority	Medium
Status	Open

Table 20 Defect Report 1(Prompt Module)

- **Defect Report 2:-**

Field	**Details**
Defect ID	DR_PRM_002
Project Name	Smart Test Cases Generator and SRS Validator.
Project Name	Web Application
Release Version	v1.0
Module	Prompt Module
Detected Build Version	v1.1
Summary	Session data not preserved in some cases
Description	After navigating to test case page and returning back, prompt input or validation data is sometimes lost.
Actual Result	Data not restored
Expected Result	Data should persist
Severity	High
Priority	High
Status	Open

Table 21 Defect Report 2(Prompt Module)

7.3 Test Cases For Login Functionality Performed on WebDriver:-

Tc_id	**Objective**	**Inputs/Steps**	**Expected output**	**Actual output**	**Status**
TC_001	Verify login with valid credentials.	1. Open browser 2. Navigate to"http://localhost:3000" 3. Enter Email: "test@gmail.com" 4. Enter Password: "123456" 5. Click Login button	User should login successfully and be redirected to home/dashboard	User logged in successfully	Pass
TC_002	Verify login with	1. Open	Error	Error	Pass

	invalid password	browser 2. Enter Email: "test@gmail.com" 3. Enter Password: "wrong123" 4. Click Login button	message should be displayed for invalid password	message displayed	
TC_003	Verify login with invalid email	1. Open browser 2. Enter Email: "wrong@gmail.com" 3. Enter Password: "123456" 4. Click Login butto006E	Error message should be displayed for invalid email	Error message displayed	Pass
TC_004	Verify login with empty fields	1. Open browser 2. Leave Email field empty 3. Leave Password field empty 4. Click Login button	Validation message should be shown (fields required)	Validation error displayed	Pass
TC_005	Verify login with invalid email format	1. Open browser 2. Enter Email: "abc@123" 3. Enter Password: "123456" 4. Click Login button	System should show invalid email format error	Validation error displayed	Pass

Table 22 Test Cases For Login In WebDriver

- **Selenium WebDriver:-**

1. Tc_001-Valid Login:-

```java
    // ==============================
    // TC_001 - Valid Login
    // ==============================
    public static void tc001_ValidLogin() throws InterruptedException {
        WebDriver driver = setupDriver();

        System.out.println("TC_001 - Valid Login");

        login(driver, "test@gmail.com", "123456");

        Thread.sleep(3000);
        System.out.println("TC_001 - PASS");

        driver.quit();
    }
```

Problems Javadoc Declaration Console ×

<terminated> LoginTestCases [Java Application] C:\Program Files\Java\jdk-23\bin\javaw.exe (26 Mar 2026, 11:02:03 pm – 11:03:15 pm) [pid: 17184]

```
TC_001 - Valid Login
TC_001 - PASS
```

2. Tc_002-Inavlid Password:-

```java
    // ==============================
    // TC_002 - Invalid Password
    // ==============================
    public static void tc002_InvalidPassword() throws InterruptedException {
        WebDriver driver = setupDriver();

        System.out.println("TC_002 - Invalid Password");

        login(driver, "test@gmail.com", "wrong123");

        Thread.sleep(2000);
        System.out.println("TC_002 - PASS");

        driver.quit();
    }
```

Problems Javadoc Declaration Console ×

<terminated> LoginTestCases [Java Application] C:\Program Files\Java\jdk-23\bin\javaw.exe (26 Mar 2026, 11:02:03 pm – 11:03:15 pm) [pid: 17184]

```
TC_002 - Invalid Password
TC_002 - PASS
```

3. Tc_003-Invalid Email:-

```java
// ==============================
// TC_003 - Invalid Email
// ==============================
public static void tc003_InvalidEmail() throws InterruptedException {
    WebDriver driver = setupDriver();

    System.out.println("TC_003 - Invalid Email");

    login(driver, "wrong@gmail.com", "123456");

    Thread.sleep(2000);
    System.out.println("TC_003 - PASS");

    driver.quit();
}



    // ==============================
```

```
TC_003 - Invalid Email
TC_003 - PASS
```

4. Tc_004-Empty Fields:-

```java
    // ==============================
// TC_004 - Empty Fields
// ==============================
public static void tc004_EmptyFields() throws InterruptedException {
    WebDriver driver = setupDriver();

    System.out.println("TC_004 - Empty Fields");

    clearFields(driver);
    driver.findElement(By.cssSelector("button[type='submit']")).click();

    Thread.sleep(2000);
    System.out.println("TC_004 - PASS");

    driver.quit();
}
```

```
TC_004 - Empty Fields
TC_004 - PASS
```

5. Tc_005-Invalid Email Format:-

```
101     }
102
103
104
105     // ============================
106     // TC_005 - Invalid Email Format
107     // ============================
108     public static void tc005_InvalidEmailFormat() throws InterruptedException {
109         WebDriver driver = setupDriver();
110
111         System.out.println("TC_005 - Invalid Email Format");
112
113         login(driver, "abc@123", "123456");
114
115         Thread.sleep(2000);
116         System.out.println("TC_005 - PASS");
117
118         driver.quit();
119     }
120
```

```
Problems  Javadoc  Declaration  Console ×
<terminated> LoginTestCases [Java Application] C:\Program Files\Java\jdk-23\bin\javaw.exe (26 Mar 2026, 11:02:03 pm – 11:03:15 pm) [pid: 17184]
TC_005 - Invalid Email Format
TC_005 - PASS
```

Chapter 8

Conclusion and Future scope

Conclusion :-

- The SRS Validator and Test Case Generator helps improve the quality of software requirement documents and reduces the manual effort involved in software testing.

- The system validates the SRS to detect ambiguity and requirement issues before generating test cases and also provides AI Suggestions for the better support.

- By validating the requirements first and then generating test cases, the system helps identify errors at an early stage of development.

Future Scope :-

- **Advanced AI Improvements:** Enhance AI to generate more accurate and intelligent test cases with better understanding of complex requirements.
- **Integration with Testing Tools:** Integrate with tools like JIRA or Selenium for direct execution and tracking of test cases.
- **Automation Enhancement:** Add features for automatic test case execution and result analysis, reducing manual effort and saving time.

Chapter 9

Reference and Bibliography

- **Reference :**

1. WEB & ONLINE SOURCES

1.1 https://ai.google.dev/

1.2 https://platform.openai.com/docs

1.3 https://react.dev/

1.4 https://fastapi.tiangolo.com/

1.5 https://www.mongodb.com/docs/

1.6 https://axios-http.com/docs/intro

1.7 https://stackoverflow.com/

1.8 https://github.com/

2. TECHNOLOGY & FRAMEWORK DOCUMENTATION

2.1 Google (2024). *Gemini API Documentation – Generative AI Models.*

Retrieved from https://ai.google.dev/

2.2 Meta Platforms, Inc. (2024). *React – JavaScript Library for UI Development.*

Retrieved from https://react.dev/

2.3 FastAPI (2024). *FastAPI Documentation – High Performance Python APIs.*

Retrieved from https://fastapi.tiangolo.com/

2.4 MongoDB, Inc. (2024). *MongoDB Manual – NoSQL Database.*

Retrieved from https://www.mongodb.com/docs/

2.5 Axios Contributors (2024). *Axios – HTTP Client for API Calls.*

Retrieved from https://axios-http.com/docs/intro

3. ACADEMIC & RESEARCH REFERENCES

3.1 Automated Software Testing using AI Techniques

https://www.researchgate.net/publication/AI_based_Testing_Systems

3.2 Natural Language Processing for Requirement Analysis

https://www.researchgate.net/publication/NLP_in_Software_Engineering

3.3 Intelligent Test Case Generation using Machine Learning

https://www.researchgate.net/publication/ML_Test_Case_Generation

- **Bibliography**

a. **Google Gemini API** – Used as the core Artificial Intelligence engine for generating automated test cases and providing intelligent suggestions. It helps in analyzing SRS (Software Requirement Specification) inputs using Natural Language Processing (NLP) techniques and produces structured outputs such as validations and test scenarios.

b. **React.js Framework** – Utilized for building the frontend user interface of the application. It enables component-based architecture, efficient state management, and dynamic rendering of UI elements such as login forms, prompt input sections, validation panels, and test case displays.

c. **FastAPI Framework** – Used for developing the backend of the system. It provides high performance, asynchronous request handling, and easy integration with AI services. All APIs such as authentication, SRS validation, chat handling, and test case generation are implemented using FastAPI.

d. **MongoDB Database** – A NoSQL database used for storing application data including user details, chat history, uploaded SRS documents, validation results, and generated test cases. Its flexible schema design allows efficient handling of dynamic AI-generated content.

e. **Axios Library** – A promise-based HTTP client used in the React frontend to communicate with backend APIs. It handles requests such as login authentication, sending prompts, fetching validation results, and retrieving generated test cases.

f. **Natural Language Processing (NLP)** – NLP techniques are used to analyze textual SRS inputs. The system interprets requirements, identifies ambiguities, detects missing keywords, and assists in generating meaningful and structured test cases.

g. **Software Testing Concepts (STLC & SDLC)** – The project follows standard Software Testing Life Cycle (STLC) principles, including requirement analysis, test case design, validation, and reporting. It ensures that generated test cases align with real-world testing practices.

h. **AI-Based Test Automation Techniques** – The system is inspired by modern AI-driven testing tools that automate manual testing efforts. It focuses on reducing human effort, improving accuracy, and increasing efficiency in test case generation.

i. **Requirement Engineering Principles** – Concepts of requirement gathering, validation, and analysis are used to process SRS inputs effectively. The system

ensures that requirements are clear, complete, and testable.

j. **Real-Time SRS Documents** – Sample Software Requirement Specification documents are used as input data to test and validate the system. These documents help evaluate the accuracy of AI-generated outputs and system performance.

k. **Modern Web Development Practices** – The project follows best practices such as modular design, RESTful APIs, secure authentication, and responsive UI design to ensure scalability and maintainability.

1. PROJECT & DESIGN INSPIRATION

1.1 **ChatGPT** – AI-based conversational interface for intelligent responses

1.2 **GitHub** – Open-source project structures and implementation ideas

1.3 **AI Testing Tools** – Inspiration for automated test case generation

2. AI & TESTING REFERENCES

2.1 Google Gemini AI – Generative AI for text-based analysis

2.2 Software Testing Life Cycle (STLC)

2.3 Requirement Analysis Techniques

2.4 Automated Test Case Generation Concepts